

Kundsegmentering med RFM och K-Means

Teknisk rapport, del 2 – projektuppgift

Grupp: Amir Hamza Jafari, Andreas Malmgren, Robin Hellgren

Repo: <https://github.com/amirhamza247/ai-customer-segmentation>

1. Bakgrund

Företag vill veta vilka kunder som är mest värdefulla, vilka som håller på att försvinna och vilka som bara handlat en gång. Då kan de rikta erbjudanden och kampanjer till rätt grupp. Vi ville bygga ett verktyg som gör den analysen automatiskt: man laddar upp en CSV med transaktioner och får tillbaka kundsegment som går att förstå.

Vi valde datasetet **Online Retail II** (UCI, via Kaggle). Det innehåller alla transaktioner från en brittisk e-handlare mellan 2009-12-01 och 2011-12-09, totalt drygt 1 miljon rader. Varje rad är en producerad på en faktura med kolumnerna Invoice, Quantity, InvoiceDate, Price och Customer ID. Datan räcker för vårt mål, eftersom det är precis de fält som behövs för en RFM-analys:

- **Recency:** antal dagar sedan kundens senaste köp (lägre är bättre).
- **Frequency:** antal unika fakturor.
- **Monetary:** total summa ($\text{Quantity} \times \text{Price}$).

Segmenteringen görs med **K-Means**, en oövervakad klustringsalgoritm. Den passar eftersom datan saknar färdiga etiketter som "lojal kund", och den är enkel att förklara.

Vi bestämde tidigt att projektet skulle vara en **proof of concept** med fokus på att hela flödet fungerar från uppladdning till resultat. Därför begränsade vi oss till ett känt CSV-format och till RFM-baserad segmentering (inga churn- eller LTV-modeller).

2. Flödet i applikationen

```
CSV-uppladdning → validering → rensning → RFM → log1p + StandardScaler  
→ jämför K = 2–8 med silhouette → användaren väljer K (2–4) → K-Means  
→ klustersammanfattning och beskrivningar → spridningsdiagram → AI-förklaring (valfri)
```

1. **Validering och rensning.** Filen läses som text så att ogiltiga värden kan hanteras kontrollerat. Rader tas bort om kund-ID saknas, fakturan är en makulering (börjar på C), datumet är ogiltigt eller antal/pris är saknat, oändligt, noll eller negativt. Appen visar hur många rader som tagits bort av varje orsak. Dubbeltrader behålls med avsikt, eftersom de kan vara riktiga köp.
2. **RFM.** Transaktionerna grupperas per kund. Referensdatumet är dagen efter den sista transaktionen i datan, inte dagens datum, så att resultatet blir detsamma varje gång man kör.
3. **Förbehandling.** Frequency och Monetary är mycket sneda (några få kunder köper för hundratusentals pund), så de log-transformeras med $\log_{10}$. Sedan standardiseras alla tre med `StandardScaler` så att ingen variabel dominerar avståndsberäkningen bara för att den har större enheter. Originalvärdena sparas för att visa och tolka resultatet.
4. **Val av K.** Appen tränar tillfälliga K-Means-modeller för $K = 2-8$ och räknar silhouette score för alla kunder. Det bästa K:et föreslås, men användaren väljer själv mellan 2, 3 och 4 kluster.
5. **Klustring och tolkning.** Den slutliga modellen tränas med `n_init=10` och `random_state=42`. Varje kluster sammanfattas med antal kunder och medelvärden i originalenheter. En regelbaserad beskrivning jämför klustrets medelvärde med snittkunden, till exempel *More recent / Higher frequency / Higher spending*.
6. **Visualisering.** Interaktiva spridningsdiagram (Recency mot Monetary) med linjär och logaritmisk axel. Man kan välja att dölja den översta procenten av extremkunder i diagrammet utan att de påverkar klustringen.
7. **AI-förklaring (valfri).** Klustersammanfattningen, alltså inga kund-ID:n eller transaktioner, skickas till en språkmodell via Groq. Den svarar med en kort förklaring på engelska om vilka kluster som står för

mest intäkter. Andelar och medelvärden räknas ut i Python innan de skickas, eftersom språkmodeller ofta räknar fel.

3. Huvudresultat

Rensning. Av 1 067 371 rader behölls 805 549. De som togs bort var 243 007 rader utan kund-ID, 18 744 makuleringar och 71 rader med pris 0. Kvar blev **5 878 kunder**, med referensdatum 2011-12-10.

Kunddaten är mycket sned. Mediankunden har handlat 3 gånger för 899 pund, men medelvärdet är 3 019 pund och den största kunden har handlat för 608 822 pund. Det bekräftade att log-transformering behövdes före klustringen.

Val av K.

K	2	3	4	5	6	7	8
Silhouette	0,419	0,401	0,361	0,367	0,349	0,336	0,317

K = 2 får högst poäng, men K = 3 ligger nästan lika högt och ger en mer användbar uppdelning för ett företag. Därför låter vi användaren välja. Resultatet med **K = 3**:

Kluster	Beskrivning	Kunder	Recency (dagar)	Frequency	Monetary (£)	Andel av intäkter
1	Nyligen / ofta / hög köpsumma	1 689 (29 %)	58	15,9	8 691	82,7 %
2	Nyligen / sällan / låg köpsumma	2 352 (40 %)	93	2,9	861	11,4 %
0	Länge sedan / sällan / låg köpsumma	1 837 (31 %)	474	1,8	566	5,9 %

Det tydligaste resultatet är att **29 % av kunderna står för 83 % av intäkterna**, vilket stämmer med Paretoprincipen. Kluster 0 är kunder som inte handlat på över ett år och troligen redan har lämnat. Kluster 2 är aktiva men handlar sällan. Det är den grupp där det kan löna sig mest att försöka få kunderna att komma tillbaka. Med K = 4 delas de bästa kunderna upp ytterligare: 952 kunder (16 %) står då för 71 % av intäkterna.

Begränsningar. En silhouette score runt 0,4 betyder att klustren är tydliga men överlappar en del. Kunddata bildar sällan helt separata grupper. Beskrivningarna säger bara om ett kluster ligger över eller under snittet, inte hur mycket. Vi har därför valt att inte ge klustren säljiga namn som "VIP", eftersom det kunde överdriva vad datan faktiskt visar.

4. Teknisk specifikation

Del	Teknik	Användning
Språk och miljö	Python 3.14, uv	Beroenden läses i <code>uv.lock</code> , så alla i gruppen får samma versioner
Databehandling	pandas , NumPy	Inläsning, rensning, RFM med <code>groupby().agg()</code>
ML	scikit-learn	StandardScaler, KMeans, <code>silhouette_score</code>
Frontend	Streamlit	Uppladdning, tabeller, val av K, Vega-Lite-diagram
AI-förklaring	Groq (openai/gpt-oss-120b)	API-nyckeln ligger i <code>.streamlit/secrets.toml</code> , som inte checkas in
Kodkvalitet	Ruff, Black	Lint och formatering innan commit
Versionshantering	Git, GitHub	Feature-brancher, pull requests, dev och main

Koden består av tre filer. `segmentation.py` innehåller all logik för databehandling och ML som rena funktioner, en per steg i flödet. `main.py` innehåller bara Streamlit-gränssnittet. `ai_analysis.py` sköter anropet till språkmodellen. Att hålla logiken skild från gränssnittet gör det lättare att testa och förstå.

Databas. Kursen kräver att datan lagras i en databas, och vår första version gjorde det. `01_ladda_data.py` laddade CSV:n till SQLite (`data/kunder.db`), SQL-frågor i `queries.py` räknade fram RFM per kund, och den tränade modellen sparades tillsammans med segmenten i `data/app.db`. När vi byggde om till ett uppladdningsflöde kom vi fram till att databasen var negativ för just vårt projekt:

- Varje uppladdning analyseras för sig, så det finns ingen data som behöver sparas mellan körningar. Databasen blev bara en extra kopia av CSV:n.

- Flödet blev längre och svårare att följa med fyra skript som måste köras i ordning innan appen fungerar. Det blev också ett extra ställe där datan kunde bli inaktuell.
- Binära filer (.db och .pkl) i Git gick inte att granska i pull requests.

Därför behandlas datan nu i minnet med pandas, och vi använder SQLite bara om vi får ett riktigt behov av att spara data, till exempel för att jämföra segment över tid.

Agentisk kodning. Vi använde AI-kodagenter (Claude Code och Codex) för att skriva mycket av koden. För att behålla kontrollen skrev vi regelfiler (CLAUDE.md, senare AGENTS.md). De säger att arkitekturen ska vara så liten som möjligt, att inga nya beroenden får läggas till utan skäl och att varje steg ska förklaras så att vi kan stå för varje rad.

5. Utvärdering av gruppens arbete

Vad har varit bra? Vi fick ett fungerande flöde från CSV till tolkade segment tidigt, vilket var målet med PoC:n. Att dela upp arbetet i små steg (rensa, RFM, skala, välja K, klustra, namnge, visualisera) gjorde att varje del fungerade innan vi gick vidare. Standups i MEETINGS.md hjälpte oss att hålla koll på vem som gjorde vad.

Vad har vi lärt oss?

- Hur man förbereder data för klustring: varför sneda variabler behöver log-transformeras och skalas, och att silhouette score är en vägledning, inte ett facit.
- Att K-Means kluster-ID:n inte betyder något i sig. Tolkningen måste komma från statistiken för varje kluster.
- Att arbeta agentiskt: AI-agenter skriver kod snabbt, men utan tydliga regler blir koden större än den behöver vara. Vi lärde oss att styra agenten med regelfiler, granska diffen innan varje commit och kräva förklaringar av det vi inte förstod. Vi provade också att följa agentarbetet tillsammans via Live Share.

Hur har Git och GitHub fungerat? Vi arbetade med feature-brancher per person och uppgift (till exempel kmeans-modell-train-robin, poc-streamlit-app-andreas, ai-segmentation-with-codex-amir) och slog ihop dem via pull requests till dev/main. Totalt har vi slagit ihop ett tiotal PR:er. Första dagen övade vi PR-flödet med testfiler, vilket gjorde att alla kom igång. Det som fungerade sämre var att två parallella versioner av appen växte fram på olika brancher (SQLite-skript respektive uppladdningsflödet). Det krävde en omstart och merge-arbete. Vi hade också commit-meddelanden på både svenska och engelska och olika Git-identiteter för samma person.

Vad hade vi gjort annorlunda?

- Bestämt arkitekturen (uppladdning eller databas) tillsammans *innan* vi började koda, så att vi inte byggt två versioner parallellt.
- Kommit överens om branch-strategi, språk i commits och code review från start. Code review-delen i MEETINGS.md användes knappt.
- Skrivit några enkla automatiska tester för rensningen och RFM-beräkningen tidigt.
- Satsat på att driftsätta appen (till exempel Streamlit Community Cloud) för att få en publik länk.